



**INSTITUTO DE EDUCACIÓN SECUNDARIA LA MOLA
NOVELDA**

DESARROLLO DE APLICACIONES WEB

Curso Académico 2025/2026

Desarrollo Web en Entorno Cliente

**U04_A04 Casos prácticos, Ponte a prueba, etc... -
Prof. Emilio Ramírez**

Autor: Segura Abad, Mario

Fecha: 11 de Diciembre, 2025

ÍNDICE

INTRODUCCIÓN..... 3

ENUNCIADO 4

PROBAD TODO EL CÓDIGO DEL APARTADO 14

CASO PRÁCTICO 17

Un cronómetro sencillo7

PONTE A PRUEBA 18

Errores en el trabajo con callbacks8

CLAVES DE RESOLUCIÓN8

DESARROLLO DE LA PRÁCTICA 9

DESCRIPCIÓN9

 • *Crear un cronómetro sencillo que muestre segundos transcurridos en una página web (Caso práctico 1).9*

 • *Analizar y corregir en el uso de callbacks y del objeto this (Ponte a prueba 1).9*

CONTENIDO DEL ARCHIVO9

Código del Apartado 1 de teoría.....9

Caso Práctico 1 – Un cronómetro12

Ponte a Prueba 1 – Errores en el trabajo con callbacks13

CÓMO VISUALIZARLO14

Requisitos:14

Pasos:.....14

OBJETIVOS DE LA PRÁCTICA14

CONCLUSIÓN..... 15

INTRODUCCIÓN

En el ejercicio “Probad todo el código del Apartado 1” deberemos incluir en uno o varios fuentes (yo siempre recomiendo varios) absolutamente todo el código que aparece en la teoría del apartado 1 de programación asíncrona, así como su ejecución y prueba de autoría.

En el “Caso práctico 1” deberemos hacer un cronómetro sencillo. Deberemos utilizar la función `setInterval()` para crear un cronómetro que nos muestre el número de segundos que transcurren en una página web.

En el “Ponte a prueba 1” deberemos arreglar el código adjunto en el ejercicio en aules cuyo funcionamiento es erróneo. Explicaremos, asimismo, por qué no funciona bien.

ENUNCIADO

Probad todo el código del Apartado 1

Incluid en uno o varios fuentes **todo** el código que aparece en la teoría de este apartado, así como su ejecución y prueba de autoría.

Ejemplo de implementación:

Captura de pantalla – DWEC_U04_A01_TEO_1.js

```
JS DWEC_U04_A01_TEO_1.js ×  
JS DWEC_U04_A01_TEO_1.js > ...  
1 // Autor: Mario Segura Abad  
2 // Fecha: 05/12/2025  
3  
4  
5 // 1.1. Funcionamiento básico  
6 let letras = ["a", "b", "c"];  
7  
8 let letrasMayusculas = letras.map(function(letra) {  
9   console.log(`Convirtiendo letra '${letra}'`);  
10  return letra.toUpperCase();  
11 });  
12  
13 console.log("Esta línea se ejecuta después de que termine 'map'");
```

Captura de pantalla – DWEC_U04_A01_TEO_2.js

```
JS DWEC_U04_A01_TEO_2.js ×  
JS DWEC_U04_A01_TEO_2.js > ...  
1 // Autor: Mario Segura Abad  
2 // Fecha: 05/12/2025  
3  
4  
5 // 1.2. Programación asíncrona  
6 function saludar() {  
7   console.log("hola");  
8 }  
9  
10 // Mostrará "hola" transcurridos dos segundos  
11 setTimeout(saludar, 2000);
```

Captura de pantalla – DWEC_U04_A01_TEO_3.js

```
JS DWEC_U04_A01_TEO_3.js ×
JS DWEC_U04_A01_TEO_3.js > ...
1  // Autor: Mario Segura Abad
2  // Fecha: 05/12/2025
3
4
5  // 1.2. Programación asíncrona
6  function saludar() {
7    |   console.log("hola");
8    | }
9
10 // Mostrará "hola" transcurridos dos segundos
11 setTimeout(saludar, 2000);
12
13 console.log("Esta línea se ejecutará ANTES de que se muestre por pantalla 'hola'");
14
15 // Mostrará "hola" transcurridos dos segundos
16 setTimeout(function() {
17 |   console.log("hola");
18 | }, 2000);
19
20 // Mostrará "adiós" transcurridos tres segundos
21 setTimeout(function() {
22 |   console.log("adiós");
23 | }, 3000);
```

Captura de pantalla – DWEC_U04_A01_TEO_4.js

```
JS DWEC_U04_A01_TEO_4.js > ...
1  // Autor: Mario Segura Abad
2  // Fecha: 05/12/2025
3
4
5  // 1.3. Inconvenientes de los "callbacks"
6  setTimeout(function() {
7    |   console.log("Primer mensaje después de 2 segundos.");
8    |   // Ponemos en marcha el segundo temporizador
9    |   setTimeout(function() {
10 |     |   console.log("Segundo mensaje, 3 segundos después del primero.");
11 |     |   // Ponemos en marcha el tercer temporizador
12 |     |   setTimeout(function() {
13 |     |     |   console.log("Tercer mensaje, 1 segundo después del segundo.");
14 |     |     | }, 1000);
15 |     |   }, 3000);
16 |   }, 2000);
```

Captura de pantalla – DWEC_U04_A01_TEO_5.js

```
JS DWEC_U04_A01_TEO_5.js > ...
1  // Autor: Mario Segura Abad
2  // Fecha: 05/12/2025
3
4
5  // 1.3. Inconvenientes de los "callbacks"
6  var resultado = "";
7
8  setTimeout(function() {
9    |   resultado = "hola";
10 | }, 2000);
11
12 console.log(resultado); // Cadena vacía: 'resultado' se actualiza cuando se ejecute el callback
```

Captura de pantalla – DWEC_U04_A01_TEO_6.js

```
JS DWEC_U04_A01_TEO_6.js X
JS DWEC_U04_A01_TEO_6.js > ...
1 // Autor: Mario Segura Abad
2 // Fecha: 05/12/2025
3
4
5 // 1.3. Inconvenientes de los "callbacks"
6 let usuario = {
7   nombre: "Laura",
8   saludarConRetardo: function() {
9     setTimeout(function() {
10       // Función convencional: define 'this', que se ejecuta en el contexto
11       // de una función anónima
12       // (no tiene referencia ni implícita ni explícita al objeto sobre el que se ejecuta).
13       // 'this' en este caso apunta al objeto global
14       console.log(`Hola, soy ${this.nombre}`);
15     }, 2000);
16   }
17 };
18
19 usuario.saludarConRetardo();
20 // Cuando pasen dos segundos se mostrará: 'Hola, soy undefined'
```

Captura de pantalla – DWEC_U04_A01_TEO_7.js

```
JS DWEC_U04_A01_TEO_7.js X
JS DWEC_U04_A01_TEO_7.js > ...
1 // Autor: Mario Segura Abad
2 // Fecha: 05/12/2025
3
4
5 // 1.3. Inconvenientes de los "callbacks"
6 let usuario = {
7   nombre: "Laura",
8   saludarConRetardo: function() {
9     setTimeout(() => {
10       // Función flecha: no tiene 'this'
11       // Por tanto, 'this' se coge del contexto léxico
12       // (cuerpo de la función 'saludarConRetardo')
13       console.log(`Hola, soy ${this.nombre}`);
14     }, 2000);
15   }
16 };
17
18 usuario.saludarConRetardo();
19 // Cuando pasen dos segundos se mostrará: 'Hola, soy Laura'
```

Caso práctico 1

Un cronómetro sencillo

La función **setInterval** funciona de manera parecida a **setTimeout**, pero con la diferencia de que ejecuta la función *callback* de manera periódica cada intervalo de tiempo. Así, el código **let temp1 = setInterval(miFuncion, 1000)** ejecutará **miFuncion()** cada segundo. Para detener su ejecución hay que utilizar **clearInterval** indicando como parámetro la variable que apunta al temporizador: en el caso del ejemplo, sería **clearInterval(temp1)**.

Utilizando **setInterval**, crea un cronómetro que muestre el número de segundos transcurridos en una página web. Debe incorporar un botón de inicio/parada y un botón de puesta a cero.

Ejemplo de implementación:

Captura de pantalla – CasoPractico1.html

```
<> CasoPractico1.html X
CasoPractico > <> CasoPractico1.html > ...
1  <!-- Autor: Mario Segura Abad -->
2  <!-- Fecha: 09/12/2025 -->
3
4  <!DOCTYPE html>
5  <html lang="es">
6    <head>
7      <meta charset="utf-8" />
8      <title>Cronómetro sencillo (corregido)</title>
9      <meta name="viewport" content="width=device-width,initial-scale=1" />
10   </head>
11   <body>
12     <h1>Cronómetro</h1>
13
14     <div id="tiempo">0</div>
15
16     <button id="inicioParar">Iniciar</button>
17     <button id="reset">Poner a cero</button>
18
19     <script>
20       // Esperamos a que el DOM esté listo
21       window.addEventListener('DOMContentLoaded', () => {
22
23         // Referencias a elementos
24         const pantalla = document.getElementById('tiempo');
25         const btnInicioParar = document.getElementById('inicioParar');
26         const btnReset = document.getElementById('reset');
27
28         // Estado del cronómetro
29         let segundos = 0;
30         let intervalo = null; // guardará el id devuelto por setInterval
31         let enMarcha = false;
32
33         // Actualiza la pantalla con el valor actual
34         function actualizarPantalla() {
35           pantalla.textContent = segundos;
36         }
37
38         // Función para iniciar el intervalo
39         function iniciarCrono() {
40           if (intervalo !== null) return; // ya iniciado (protección)
41           intervalo = setInterval(() => {
42             segundos++;
```

Ponte a prueba 1

Errores en el trabajo con *callbacks*

El código que se proporciona a continuación no funciona bien. ¿Por qué?
¿Cómo podrías arreglarlo?

```
let temporizador = {
  cuentaAtras: () => {
    setTimeout(function() {
      console.log(`Ha terminado el temporizador de
        ${this.tiempo} ms.`);
    }, this.tiempo);
  }
};

temporizador.tiempo = 2000;
temporizador.cuentaAtras();
```

Ejemplo de implementación:

Captura de pantalla – PonteAPrueba1.js



```
JS PonteAPrueba1.js X
PonteAPrueba > JS PonteAPrueba1.js > ...
1 // Autor: Mario Segura Abad
2 // Fecha: 09/12/2025
3
4 let temporizador = {
5   cuentaAtras() { // método normal: this -> temporizador
6     setTimeout(() => { // arrow -> hereda este this (temporizador)
7       console.log(`Ha terminado el temporizador de ${this.tiempo} ms.`);
8     }, this.tiempo);
9   }
10 };
11
12 temporizador.tiempo = 2000;
13 temporizador.cuentaAtras(); // OK -> muestra: Ha terminado el temporizador de 2000 ms.
```

Claves de resolución

- Ejecuta el código en la consola del navegador o en NodeJS:
 - ¿Se produce el tiempo de espera?
 - ¿Se muestra el texto correcto?
- Observa cómo están definidas las funciones **cuentaAtras** y el *callback* de **setTimeout**.
- Recuerda cómo funciona **this** en las funciones JavaScript.

DESARROLLO DE LA PRÁCTICA

Práctica DWEK – Unidad 4 Apartado 1

Descripción

Este apartado se centra en el uso de **temporizadores en JavaScript** (`setInterval` y `setTimeout`) para ejecutar funciones de manera periódica.

El objetivo principal ha sido:

- Crear un cronómetro sencillo que muestre segundos transcurridos en una página web (Caso práctico 1).
- Analizar y corregir en el uso de callbacks y del objeto `this` (Ponte a prueba 1).

Contenido del archivo

Código del Apartado 1 de teoría

Se ha probado todo el código relacionado con:

- Uso de `setInterval` para ejecutar funciones periódicamente.
- Uso de `clearInterval` para detener la ejecución.
- Uso de `setTimeout` para ejecutar funciones tras un tiempo determinado.
- Diferencias entre funciones normales y funciones flecha respecto al contexto de `this`.

Cada archivo incluye comentarios con el nombre del autor y la fecha de realización.

Se han capturado los resultados por consola y pantalla como prueba de ejecución.

Ejemplo de ejecución:

Salida por consola – DWEK_U04_A01_TEO_1.js

```
// Prueba de autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 05/12/2025");
Convirtiendo letra 'a'
Convirtiendo letra 'b'
Convirtiendo letra 'c'
Esta línea se ejecuta después de que termine 'map'
Autor: Mario Segura Abad
Fecha de realización: 05/12/2025
```

⏪ undefined

Salida por consola – DWEK_U04_A01_TEO_2.js

```
// Prueba de Autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 05/12/2025");
Autor: Mario Segura Abad
Fecha de realización: 05/12/2025
< undefined
hola
```

Salida por consola – DWEK_U04_A01_TEO_3.js

```
// Prueba de Autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 05/12/2025");
Esta línea se ejecutará ANTES de que se muestre por pantalla 'hola'
Autor: Mario Segura Abad
Fecha de realización: 05/12/2025
< undefined
hola
hola
adiós
```

Salida por consola – DWEK_U04_A01_TEO_4.js

```
// Prueba de Autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 05/12/2025");
Autor: Mario Segura Abad
Fecha de realización: 05/12/2025
< undefined
Primer mensaje después de 2 segundos.
Segundo mensaje, 3 segundos después del primero.
Tercer mensaje, 1 segundo después del segundo.
```

Salida por consola – DWEC_U04_A01_TEO_5.js

```
// Prueba de autoría  
console.log("Autor: Mario Segura Abad");  
console.log("Fecha de realización: 05/12/2025");
```

Autor: Mario Segura Abad

Fecha de realización: 05/12/2025

⏪ undefined

Salida por consola – DWEC_U04_A01_TEO_6.js

```
// Prueba de Autoría  
console.log("Autor: Mario Segura Abad");  
console.log("Fecha de realización: 05/12/2025");
```

Autor: Mario Segura Abad

Fecha de realización: 05/12/2025

⏪ undefined

Hola, soy undefined

Salida por consola – DWEC_U04_A01_TEO_7.js

```
// Prueba de Autoría  
console.log("Autor: Mario Segura Abad");  
console.log("Fecha de realización: 05/12/2025");
```

Autor: Mario Segura Abad

Fecha de realización: 05/12/2025

⏪ undefined

Hola, soy Laura

Caso Práctico 1 – Un cronómetro

Objetivo:

- Crea un cronómetro que muestre el número de segundos transcurridos en la página.
- Incorporar un botón de **inicio/parada** y un botón de **puesta a cero**.

Claves de resolución:

- Usar `setInterval` para incrementar el contador cada segundo.
- Guardar el identificador del intervalo en una variable para poder detenerlo con `clearInterval`.
- Reiniciar el contador a cero al pulsar el botón correspondiente.

Este ejercicio ha reforzado el uso de **temporizadores periódicos** y el control de ejecución con botones en la interfaz.

Ejemplo de ejecución:

Salida por consola – CasoPractico1.html

Cronómetro

0

Iniciar

Poner a cero

Autor: **Mario Segura Abad**

Fecha de realización: 09/12/2025

Cronómetro

25

Parar

Poner a cero

Ponte a Prueba 1 – Errores en el trabajo con callbacks

Código inicial (erróneo):

```
let temporizador = {
  cuentaAtras: () => {
    setTimeout(function() {
      console.log(`Ha terminado de ${this.tiempo} ms.`);
    }, this.tiempo);
  }
};
temporizador.tiempo = 2000;
temporizador.cuentaAtras();
```

Problema:

- La función cuentaAtras está definida como **función flecha**, por lo que this no apunta al objeto temporizador.
- En el callback de setTimeout, this tampoco hace referencia al objeto esperado.

Solución:

- Definir cuentaAtras como función normal para que this apunte al objeto.
- O bien capturar la referencia con una variable (let self = this).

Código corregido:

```
let temporizador = {
  cuentaAtras: function() {
    setTimeout(() => {
      console.log(`Ha terminado el temporizador de ${this.tiempo} ms.`);
    }, this.tiempo);
  }
};

temporizador.tiempo = 2000;
temporizador.cuentaAtras();
```

Ejemplo de ejecución:

Salida por consola – PonteAPrueba1.js

```
// Prueba de autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 09/12/2025");
Autor: Mario Segura Abad
Fecha de realización: 09/12/2025
< undefined
Ha terminado el temporizador de 2000 ms.
```

Cómo visualizarlo

Requisitos:

- Editor de código compatible: en mi caso, Visual Studio Code.
- Navegador web actualizado: en mi caso, Google Chrome.

Pasos:

1. Guardar los archivos .html y .js en la misma carpeta.
2. Abrir los archivos .html en el navegador.
3. **Caso práctico 1:** comprobar que el cronómetro inicia, se detiene y se pone a cero correctamente.
4. **Ponte a prueba 1:** ejecutar el script en consola y verificar que el mensaje se muestra tras el tiempo indicado.
5. Revisar la consola del navegador (F12 → “Consola”) para verificar mensajes de prueba de autoría.

Objetivos de la práctica

- Probar y comprender el código teórico del apartado 1.
- Implementar un cronómetro con control de inicio/parada y puesta a cero.
- Analizar y corregir errores en el uso de callbacks y this.
- Documentar correctamente, como siempre, cada ejercicio con nombre, fecha y capturas de pantalla.
- Presentar el proyecto en formato PDF junto con los archivos fuente y pantallazos.

CONCLUSIÓN

Esta práctica nos ha permitido consolidar el uso de **temporizadores en JavaScript** y comprender las diferencias entre `setInterval` y `setTimeout`. Se ha comprobado cómo crear un cronómetro funcional y cómo corregir errores relacionados con el contexto de `this`.

También nos ha sido de utilidad para entender la ejecución periódica en aplicaciones web.